CSE 151B Final Report — Team Name

Ketan Mittal
Department of Data Science
University of California, San Diego
k1mittal@ucsd.edu

Anirudh Annabathula
Department of Data Science
University of California, San Diego
aannabathula@ucsd.edu

Kaii Bijlani
Department of Data Science
University of California, San Diego
kbijlani@ucsd.edu

May 31, 2026

Abstract

This is our final report for the CSE 151B Spring 2026 Math Reasoning Competition. The data is 1,126 public math problems (375 MCQ, 751 free-form) and a 943-item private test set. The milestone got us to 0.515 on the private leaderboard with a grader-aware system prompt. For the final phase we added three methods on top of that prompt: a 24-config sampling-parameter sweep, self-consistency majority voting over k rollouts, and GRPO LoRA fine-tuning of Qwen3-4B-Thinking-2507 on a Claude-filtered subset of the public set. We also moved inference from INT8 on one GPU to BF16 with vLLM tensor-parallel = 2 on 2× A40, which roughly quadrupled throughput. The final private score is **0.699**, up +0.184 absolute and +36% relative from the milestone. The LoRA adapter is public on HuggingFace at `k1mittal/cse151b-grpo-lora`, and the whole pipeline runs from `run_inference()` in `run_inference.py`. The sampling sweep was the biggest gain, and self-consistency and GRPO were both flat on our 44-item eval slice — we kept them for robustness rather than measured gains. Free-form accuracy sat near 65% across every method we tried, and that is the bottleneck we did not move.

1 Introduction

Problem Definition. Each test item is either a multiple-choice question (MCQ) with up to ten labeled options, or a free-form question that wants one or more numeric or symbolic values. The model gets the question text (and the option list when there is one), and has to produce a final answer wrapped in a boxed expression. The provided `judger.py` then parses and scores that answer. For free-form items the judger uses a sympy-based comparator with a 10^{-8} relative tolerance; for MCQ items it just expects a single capital letter inside the box. So the task is really a constrained generation problem: the model has to do multi-step math reasoning *and* put the final answer in a specific format the judger can parse.

Problem Significance. Math reasoning is one of the things people most want LLMs to do well — tutoring tools, science assistants, and many engineering workflows assume the model can both compute the right answer and report it without ambiguity. The gap between a model that "gets" a problem internally and one that emits an answer the grading code will accept is a real, practical robustness problem, and that gap is exactly what this competition exposes.

Technical Challenge. This project is harder than it looks for a few reasons. (1) **Data:** the public set covers algebra, calculus, statistics, probability, linear algebra, geometry, number theory, and discrete math, and many of the LaTeX strings are slightly corrupted (e.g., `frac` or `int` without backslashes). Some of the published gold answers are themselves wrong or ambiguous; we use Claude as a filter to drop those before GRPO training. (2) **Grader:** the 10^{-8} *relative* tolerance means a correct-looking decimal like 0.333 is graded wrong against gold $\frac{1}{3}$; the boxed-extraction code only keeps the last contiguous group of boxed expressions after </think>. Either of these can quietly cost points. (3) **Model:** 4B parameters is small for math; the model loses the thread on harder items and restates decimals mid-solution. (4) **Engineering:** the milestone ran INT8 on one GPU. For the final phase we moved to BF16 with vLLM tensor-parallel $= 2$ on $2\times$ NVIDIA A40 (46 GB each). Qwen3-4B is only ≈ 8 GB in BF16, so quantization was mostly dequant overhead, and the BF16+TP$=2$ move roughly quadrupled throughput. (5) **Evaluation:** a full pass over 943 private items at our final $k = 3$ setting takes about four days, so we developed methods on a 44-item eval slice and submitted to the leaderboard only as a check.

Contributions. The starter notebook `starter_code_cse151b_comp.ipynb` gives an end-to-end skeleton: `uv` environment, vLLM model loading, MCQ and free-form prompts, batched generation, and scoring. The first thing we added (reported in the milestone) was a grader-aware system prompt that respects the judger's parsing rules — exact symbolic form, single boxed group — which took us to 0.515 on the private leaderboard. The rest of our work sits on top of that prompt.

- **Contribution A (milestone):** a grader-aware system prompt for both free-form and MCQ items, written around the judger's parsing rules. Pinned for everything downstream.
- **Contribution B (final):** three method layers on top of A: a 24-configuration sampling-parameter sweep, self-consistency voting over k rollouts, and GRPO [2] LoRA fine-tuning on a Claude-filtered subset of the public set. Together with the move from INT8 on one GPU to BF16 with vLLM tensor-parallel = 2 on 2×A40, these took our private score from 0.515 to 0.699.
- **Contribution C (final):** one reproducible entry point, `run_inference()` in `run_inference.py`, that loads the base model, downloads the LoRA adapter from `k1mittal/cse151b-grpo-lora` on HuggingFace Hub if it isn't already local, runs the full SC pipeline on the private set, and writes the submission CSV.

2 Related Work (Optional)

LLMs for mathematical reasoning. Open-weight "thinking" models like Qwen3-4B-Thinking-2507 [1] are trained to reason inside <think>...</think> tokens before answering. The Qwen team's evaluations recommend specific sampling settings (temp = 0.6, top-p = 0.95, top-k = 20) and a trailing user instruction ("Please reason step by step, and put your final answer within \boxed{}"). We had been using these at the milestone without ever checking them against alternatives.

GRPO and RL fine-tuning for math. Group Relative Policy Optimization (GRPO) was introduced in DeepSeekMath [2] as a way to fine-tune reasoning models with a verifier signal, and it showed that a deterministic grader as the reward function can noticeably improve a base model's math accuracy. The same recipe was used in Qwen2.5-Math [3] and again in Qwen3 [1], where the team collected around 4,000 query–verifier pairs and used GRPO after a small cold-start SFT phase. We have the judger as a 0/1 reward and the same Qwen3 backbone, so the recipe carries over, though our run came back neutral rather than positive (Section 4.4).

Prompt engineering for structured output. A recurring finding in LLM-with-grader pipelines is that spelling out the grader's parsing rules in the system prompt helps more than describing the answer's semantics alone. Our milestone prompt is an instance of this idea, specialized to the 10^{-8} relative tolerance and the contiguous-boxed-group rule.

3 Methods

Problem Setting. We treat each item as a tuple (q, O, y) where q is the question text, O is the option list (empty for free-form items), and y is the gold answer. The model defines a conditional distribution $p_\theta(r \mid s(q, O), u(q, O))$ over response strings r. The judger $\mathcal{J}(r, y, O) \in \{0, 1\}$ pulls the last contiguous group of boxed expressions from r (only after </think>), normalizes the LaTeX, and compares to y with sympy at 10^{-8} relative tolerance. We maximize $\mathbb{E}_{(q,O,y)\sim\mathcal{D}}\, \mathbb{E}_{r\sim p_\theta}[\mathcal{J}(r, y, O)]$. At the milestone we only changed s; the final phase keeps that prompt and also changes the sampling distribution, votes across samples, and updates θ through a LoRA adapter trained with GRPO.

For fast iteration we use a fixed 44-item eval slice: the first 50 items of `public.jsonl` minus six IDs ($\{0, 1, 8, 12, 13, 14\}$) whose gold answers we judged wrong or ambiguous by hand. The slice is 10 MCQ and 34 free-form. Every method below is measured on the same slice; the leaderboard is the ground truth.

Sampling parameter sweep. We swept temperature $\in \{0.3, 0.6, 0.9\}$, top_p $\in \{0.8, 0.95\}$, top_k $\in \{0, 20\}$ and min_p $\in \{0.0, 0.05\}$ — 24 configurations — with `repetition_penalty=1.0` and `max_tokens=20,000`. 3 samples per question, with per-sample seeds reused across configurations so cell-to-cell deltas are not seed luck. The winner was `(T=0.6, top_p=0.95, top_k=0, min_p=0)` at 68.18% overall (80.0% MCQ, 64.71% free-form). The Qwen-recommended `(T=0.6, top_p=0.95, top_k=20, min_p=0)` tied at 68.18%. `top_k` barely mattered, and `min_p=0.05` consistently hurt free-form. We pinned the recommended setting for everything after. This was the biggest gain in the final phase.

Self-consistency. For each item we ran $k = 8$ independent rollouts and voted: for MCQ items the modal extracted letter; for free-form items the modal judger-normalized symbolic form, so that \frac{1}{2}, \dfrac{1}{2} and 0.5 collapse into one vote. Ties broke by first occurrence. On the eval slice the vote scored 90.0 / 64.71 / 70.45 (MCQ / free-form / overall): on free-form the modal answer matched a single rollout on every item, and on MCQ one item out of ten flipped versus the best single sample (within noise on a 10-item split). Items the model got right were already high-confidence; items it got wrong tended to agree on the same wrong answer. We dropped k from 8 to 3 for the final private run — about 2.7× cheaper at the same expected accuracy on this slice.

GRPO LoRA fine-tuning. We tried GRPO [2] over a Claude-filtered subset of the public set (1,123 of 1,126 items, dropping the ones Claude flagged as wrong or ambiguous). Policy was Qwen3-4B-Thinking-2507 loaded in 4-bit nf4 with a LoRA adapter (rank 16, $\alpha = 32$, dropout 0, target modules `q_proj, k_proj, v_proj, o_proj`). The trainer was `trl`'s `GRPOTrainer` launched via `accelerate launch --num_processes=2` (DDP, one rank per GPU with `device_map` pinned to local rank, colocated vLLM rollouts). The reward was a wrapper around `Judger.auto_judge` (and an exact letter match for MCQ): +1 correct, 0 otherwise. Hyperparameters: learning rate 1×10^{-6}, KL $\beta = 0.04$, `num_generations`$=4$ (we tried 8 and ran out of memory), `max_completion_length`$=$ 2,048, per-device batch size 1 with gradient accumulation 4, one epoch (561 steps, ≈ 14 hours on $2\times$ A40). On the eval slice the GRPO adapter scored 70.45%, identical to base + SC. Probable causes: one low-LR epoch; group size 4 instead of 8–16 from the recipe, which inflates advantage variance; completions truncated at 2,048 tokens; and a sparse 0/1 reward that gave us many all-same groups with zero advantage. We kept the adapter in the final pipeline because validation showed no regression.

Two engineering issues stalled training and are worth noting. `peft 0.19` tries to import `EmbeddingParallel` from `transformers 4.57`, which no longer exports it; we aliased it to `ReplicateParallel` at import time. An NCCL timeout also silently hung training for ≈ 28 hours, so we added a log-growth watchdog that kills the process after 15 minutes with no step-advance and wrapped the loop in a resume-from-latest-checkpoint retry.

Final pipeline and a bug we caught. The submitted pipeline is self-consistency $k = 3$ + the GRPO adapter, on all 943 private items. The driver writes one JSONL record per item so it can resume, then converts to the submission CSV. On the first full run, 10 items came back empty: long reasoning traces filled the 24,576-token context window and there was zero budget left for new tokens, which vLLM rejects. We fixed `run_inference.py` to clamp `max_tokens` to the remaining context budget (and skip the item if fewer than 256 tokens are left) and re-ran those 10 items. The final CSV has 943 rows, 0 empty, 0 errors, and scored **0.699** on the private leaderboard. We also tried a longer-reasoning variant (`max_tokens` = 16,384, `max_model_len` = 49,152); it scored 0.692 on the leaderboard, slightly worse, so we did not adopt it. Qwen3-4B-Thinking-2507 supports 262,144 tokens natively, so KV cache was not the binding constraint; the free-form ceiling we are hitting is a reasoning-quality limit, not a token-budget limit.

4 Experiments

4.1 Baselines

Our baseline is the milestone pipeline: Qwen3-4B-Thinking-2507 under INT8 on a single GPU, the grader-aware system prompt, and the Qwen-recommended sampling defaults. That baseline scored 0.515 on the private leaderboard, and every later number we report is a delta on top of that. We do not include simpler models like a multi-layer perceptron or linear regression as baselines, because they cannot do free-form math reasoning over natural-language problem statements in any meaningful way. Comparing additive method changes to the same backbone is what made sense at this stage.

4.2 Evaluation

We score with the provided judger: `Judger.auto_judge` for free-form items, exact-letter match for MCQ items. For fast method development we use the 44-item eval slice from Section 3 (10 MCQ, 34 free-form). For each submitted variant we also report the Kaggle private-leaderboard score on the full 943-item private set, which we treat as ground truth.

4.3 Implementation details

- **Hardware (final):** 2× NVIDIA A40 (46 GB each); vLLM tensor-parallel = 2; BF16 weights. The milestone ran on one GPU under INT8 (BitsAndBytes); we switched because Qwen3-4B is only ≈ 8 GB in BF16 (quantization was mostly dequant overhead), and the move to BF16 with TP= 2 raised throughput from ≈ 175 to ≈ 786 output tokens/second.

- **Wall-clock:** a full 943-item private run at SC $k = 3$ takes about 4 days (≈ 10 items/hour); GRPO training was ≈ 14 hours (1 epoch, 561 steps).

- **Model:** `Qwen/Qwen3-4B-Thinking-2507` through vLLM with `dtype="bfloat16"`, `enable_prefix_caching=False`, `gpu_memory_utilization=0.75`, `max_model_len=24,576`, `max_num_seqs=128`, `max_num_batched_tokens=16,384`, `tensor_parallel_size=2`, and `enable_lora=True` when the GRPO adapter is active.

- **Sampling (final):** `temperature=0.6`, `top_p=0.95`, `top_k=20`, `min_p=0.0`, `repetition_penalty=1.0`.

- **Self-consistency:** $k = 3$ in the final pipeline ($k = 8$ tested and found neutral).

- **Reproducibility:** all of the above are locked in `run_inference.py`. `run_inference()` loads the base model, downloads the LoRA adapter from `k1mittal/cse151b-grpo-lora` if not local, runs the pipeline on `data/private.jsonl`, and writes `csv/submission.csv`.

4.4 Results

Result 1: 0.515 to 0.699 on the private leaderboard. That is +0.184 absolute or +36% relative. The sampling sweep was the biggest contribution; self-consistency and the GRPO adapter were both neutral on the eval slice (Results 2 and 3); the longer-reasoning ablation was slightly worse (Result 4).

Result 2: per-method eval-slice accuracy. Table 1 reports each stage of the pipeline on the same 44-item eval slice. The flatness of the last two rows should be read as "no signal we can resolve on a slice this small", not "we proved these methods do nothing".

Table 1: Accuracy on the 44-item eval slice. All rows use Qwen3-4B-Thinking-2507 under BF16 + vLLM TP=2 with (`T=0.6, top_p=0.95, top_k=20, min_p=0`).

Pipeline stage	MCQ ($n = 10$)	Free-form ($n = 34$)	Overall ($n = 44$)
Sampling sweep (best)	80.0%	64.7%	68.2%
+ Self-consistency ($k = 8$)	90.0%	64.7%	70.5%
+ GRPO LoRA	90.0%	64.7%	**70.5%**

Result 3 (negative): self-consistency was neutral. At $k = 8$ the vote scored 70.45% overall: identical to the best single sample on free-form, with a 1-item MCQ swing ($8/10 \rightarrow 9/10$) that we treat as noise on a 10-item split. Items the model gets right were already high-confidence; items it gets wrong tended to be wrong consistently across rollouts. We kept SC at $k = 3$ as a cheap robustness check.

Result 4 (negative): GRPO was neutral too. On the eval slice the GRPO adapter also scored 70.45%. Probable causes are listed in Section 3: one low-LR epoch, group size 4 instead of 8–16, truncated 2,048-token completions, and a sparse 0/1 reward. We kept the adapter because validation showed no regression.

Result 5 (negative): more reasoning room did not help. The longer-reasoning ablation (`max_tokens` = 16,384, `max_model_len` = 49,152) scored 0.692, slightly worse than 0.699. KV cache was not the binding constraint, so the free-form ceiling looks like a reasoning-quality limit, not a token-budget limit.

5 Discussion

We started the final phase from 0.515 on the private leaderboard and ended at 0.699 (+36% relative). The rest of this section covers what worked, what did not, and what we would try next.

Achievements. The pipeline runs end-to-end from a single function call (`run_inference()` in `run_inference.py`). The LoRA adapter is on HuggingFace Hub at `k1mittal/cse151b-grpo-lora`, so verification does not need any local model files. The full 943-item private run completed with zero empty or errored items after we caught and fixed the context-budget bug in Section 3.

Bottlenecks and Mitigation. The sampling sweep was the largest single contribution in the final phase. Self-consistency at $k = 8$ and the GRPO adapter were both neutral on our eval slice; we kept them for robustness. The persistent bottleneck is free-form accuracy, which sat at $\approx 65\%$ on the eval slice through every layer. The longer-reasoning ablation (0.692 vs. 0.699) shows the issue is not truncation. A 4B-parameter thinking model at our compute budget probably did not have enough headroom for RL to close that gap on harder derivation-style items.

Plans Forward. The clearest place to push next is the free-form ceiling.

- Better verifiers / weighted voting — use a stronger reward model to weight self-consistency votes instead of naive majority.
- A denser reward for GRPO. The 0/1 judger reward is sparse and item-level; a process- or step-level reward might give the policy more signal per group.
- More GRPO compute with larger groups (`num_generations` ≥ 8) and longer completions, to match the published recipes more faithfully than our memory-constrained single-box run.

Deviations from the milestone plan: we moved infrastructure from INT8 on one GPU to BF16 with vLLM tensor-parallel = 2 on 2× A40 for throughput; we tested SC at $k = 8$ rather than the planned $k \in \{3, 5\}$, found no gain, and shipped $k = 3$ for cost; GRPO ran with reduced group size and shorter completions because of single-box memory; we added a longer-reasoning ablation late, and it underperformed, so it was dropped.

Team Responsibilities.

- **Ketan Mittal** — grader analysis and prompt design; sampling-parameter sweep; final private run, bug-fix, and submission pipeline.
- **Anirudh Annabathula** — inference infrastructure (vLLM TP=2, BF16); GRPO training setup.
- **Kaii Bijlani** — dataset analysis; Claude-based gold-answer filter for the GRPO training set; self-consistency.

Timelines.

- **Week 2:** Formed team; reproduced the starter notebook locally.
- **Week 3:** Read `judger.py`; identified the 10^{-8} tolerance and contiguous-boxed-group behaviors.
- **Week 4:** Drafted the grader-aware system prompt on a 20-question dev subset; first private submission (0.515).
- **Week 5:** Submitted milestone report.
- **Week 6:** Moved inference to BF16 + vLLM TP=2 on 2×A40; ran the 24-config sampling sweep.

- **Week 7:** Ran self-consistency at $k = 8$ on the eval slice and found it neutral.
- **Week 8:** Built the Claude-based gold-answer filter; produced `public_filtered.jsonl`.
- **Week 9:** Ran GRPO LoRA training ($\approx$ 14 hours, 561 steps); fixed two engineering issues; uploaded the adapter to `k1mittal/cse151b-grpo-lora`.
- **Week 10:** Full 943-item private run with SC $k = 3$ + GRPO adapter; caught and fixed the context-budget bug; submitted 0.699; ran the longer-reasoning ablation (0.692, not submitted); wrote up the final report.

LLM Usage

We used an LLM assistant (Anthropic Claude) for two things in the report-writing process: (i) help with wording, mostly tightening up phrasing in this report and in our system prompts, and (ii) help making sense of hard-to-read error messages in LaTeX and Python while we were writing the report and building the pipeline. We also used Claude as a *verifier* for the GRPO training set — it read each public-set item and flagged ones whose published gold answer was wrong or ambiguous — which is methodological use rather than report-writing assistance, and is described as such in Section 3. All the technical decisions, code, experimental runs, and numbers in this report are ours.

References

[1] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL https://arxiv.org/abs/2505.09388.

[2] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.

[3] An Yang et al. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024. URL https://arxiv.org/abs/2409.12122.